



Hệ sinh thái Hadoop

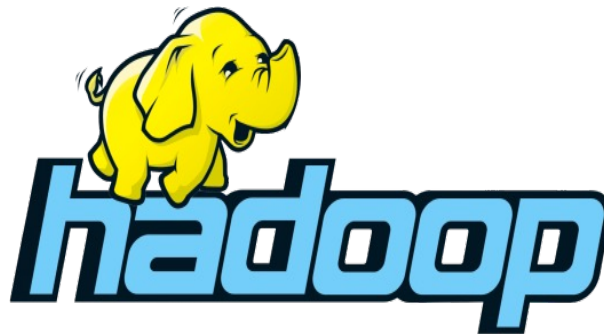
ONE LOVE. ONE FUTURE.

Nội dung

- Apache Hadoop
- Hệ thống tệp tin Hadoop (HDFS)
- Mô thức xử lý dữ liệu MapReduce
- Các thành phần khác trong hệ sinh thái Hadoop

Mục tiêu của Hadoop

- Mục tiêu chính
 - Lưu trữ dữ liệu lớn, tin cậy
 - Xử lý dữ liệu hiện quả
- Với thách thức
 - Thiết bị lưu trữ tốc độ chậm, máy tính thiếu tin cậy, lập trình song song phân tán không dễ dàng

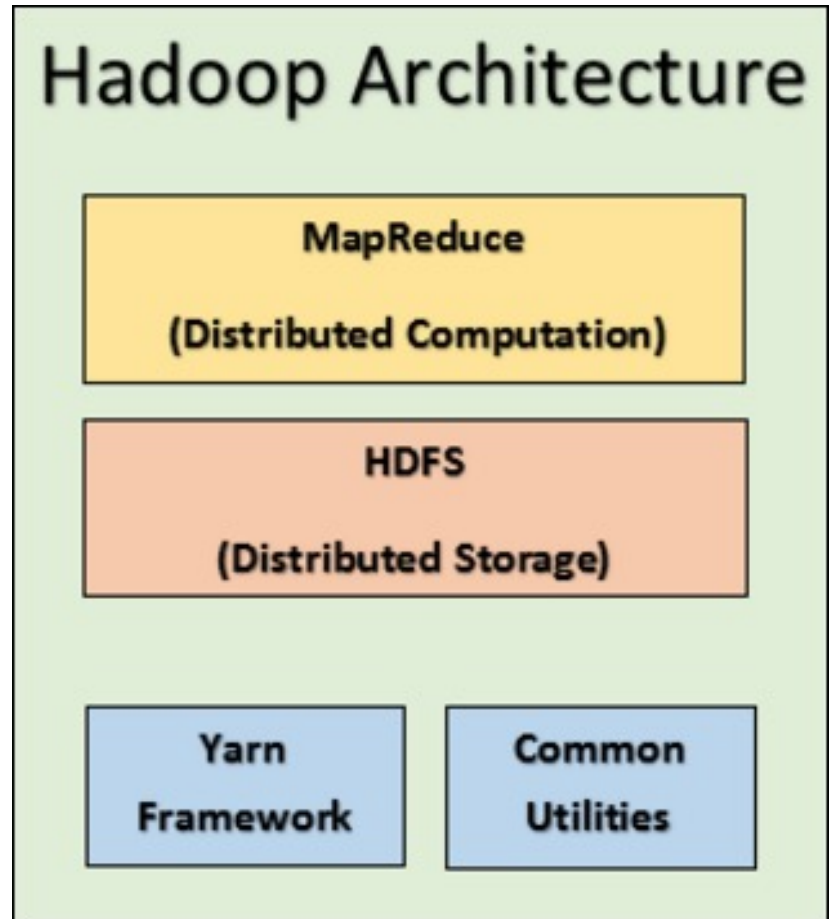


Giới thiệu về Apache Hadoop

- Apache Hadoop: framework
 - Tập hợp các thư viện mã nguồn mở
 - Xử lý phân tán dữ liệu lớn thông qua các cụm cluster một cách hiệu quả
- Lưu trữ và xử lý dữ liệu hiệu quả
 - Thiết kế để vận hành trên phần cứng phổ thông, có khả năng chống chịu lỗi phần cứng
 - Hadoop thiết kế để mở rộng thông qua kỹ thuật scale-out, tăng số lượng máy chủ
 - Xử lý dữ liệu phân tán với mô hình lập trình đơn giản, thân thiện hơn như MapReduce
 - Lấy cảm hứng từ kiến trúc dữ liệu của Google

Các thành phần chính của Hadoop

- Lưu trữ dữ liệu: Hệ thống tệp tin phân tán Hadoop (HDFS)
- Xử lý dữ liệu: MapReduce framework
- YARN: framework quản lý tài nguyên và lập lịch trong cụm Hadoop.
- Hadoop Common: Các tiện ích chung hỗ trợ các thành phần của Hadoop.



Hadoop giải quyết bài toán khả mở

- Thiết kế hướng “phân tán” ngay từ đầu
 - Hadoop mặc định thiết kế để triển khai trên cụm máy chủ
- Các máy chủ tham gia vào cụm được gọi là các Nodes
 - Mỗi node tham gia vào cả 2 vai trò lưu trữ và tính toán
- Hadoop mở rộng bằng kỹ thuật scale-out
 - Có thể tăng cụm Hadoop lên hàng chục ngàn nodes

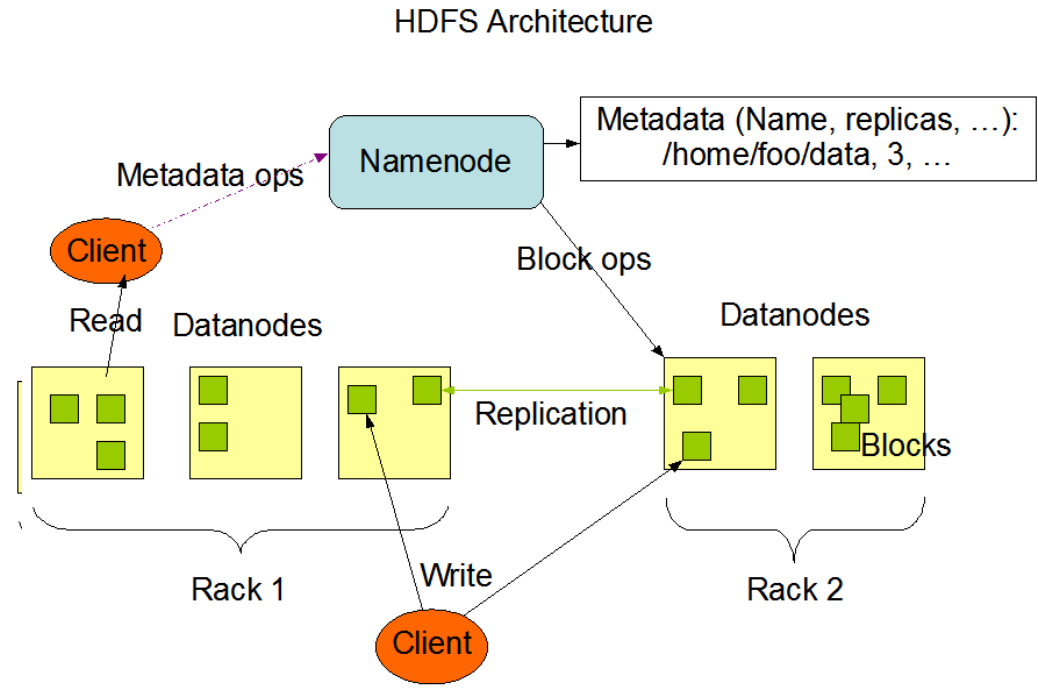
Hadoop giải quyết bài toán chịu lỗi

- Với việc triển khai trên cụm máy chủ phổ thông
 - Hồng hóc phần cứng là chuyện thường ngày, không phải là ngoại lệ
 - Hadoop chịu lỗi thông qua kỹ thuật “dư thừa”
- Các tệp tin trong HDFS được phân mảnh, nhân bản ra các nodes trong cụm
 - Nếu một node gặp lỗi, dữ liệu ứng với nodes đó được tái nhân bản qua các nodes khác
- Công việc xử lý dữ liệu được phân mảnh thành các tác vụ độc lập
 - Mỗi tác vụ xử lý một phần dữ liệu đầu vào
 - Các tác vụ được thực thi song song với các tác vụ khác
 - Tác vụ lỗi sẽ được tái lập lịch thực thi trên node khác
- Hệ thống Hadoop thiết kế sao cho các lỗi xảy ra trong hệ thống được xử lý tự động, không ảnh hưởng tới các ứng dụng phía trên

- HDSF: Hadoop Distributed File System
- Cung cấp khả năng lưu trữ tin cậy và chi phí hợp lý cho khối lượng dữ liệu lớn
- Tối ưu cho các tập tin kích thước lớn (từ vài trăm MB tới vài TB)
- HDFS có không gian cây thư mục phân cấp như UNIX (vd., /hust/soict/hello.txt)
- Khác biệt so với hệ thống tập tin trên UNIX
 - Chỉ hỗ trợ thao tác ghi thêm dữ liệu vào cuối tệp (APPEND)
 - Ghi một lần và đọc nhiều lần (write-once-read-many access model)

HDFS

- Kiến trúc Master/Slave
- HDFS master: name node
 - Quản lý không gian tên và siêu dữ liệu ánh xạ tệp tin tới vị trí các chunks
 - Giám sát các data node
- HDFS slave: data node
 - Trực tiếp thao tác I/O các chunks



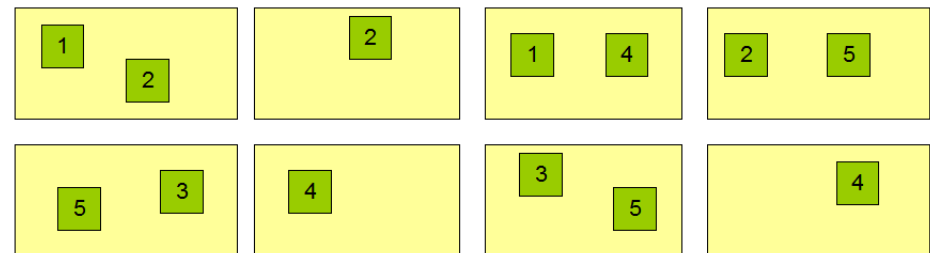
HDFS

- I/O pattern
 - Chỉ ghi thêm (Append) -> giảm chi phí điều khiển tương tranh
- Phân tán dữ liệu
 - Tập được chia thành các chunks lớn (64 MB, 128 MB)
 - > Giảm kích thước metadata
 - > Giảm chi phí truyền dữ liệu
- Nhân bản dữ liệu
 - Mỗi chunk thông thường được sao làm 3 nhân bản

Block Replication

Namenode (Filename, numReplicas, block-ids, ...)
/users/sameerp/data/part-0, r:2, {1,3}, ...
/users/sameerp/data/part-1, r:3, {2,4,5}, ...

Datanodes



MapReduce

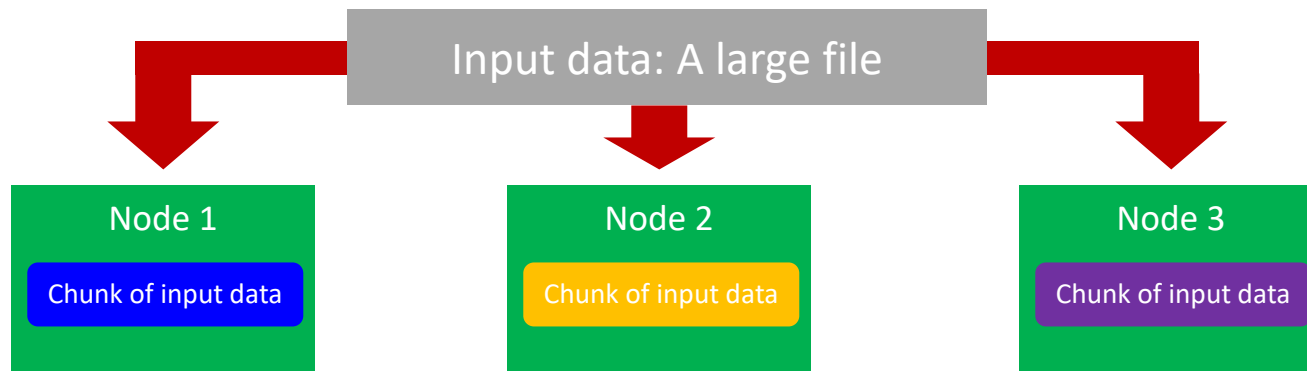
- MapReduce: software framework cho việc xử lý dữ liệu lớn một cách song song trên các cụm máy tính một cách hiệu quả
 - Dữ liệu: TB ($=10^3\text{GB}$), PB ($=10^3\text{TB}$)
 - Cụm: hàng nghìn node
 - Node tính toán và nút dữ liệu có thể là một $\Rightarrow$ HDFS và MapReduce có thể chạy trên cùng node
- Đặc điểm của MapReduce
 - Đơn giản (Simplicity)
 - Linh hoạt (Flexibility)
 - Khả mở (Scalability)

A MapReduce job = {Isolated Tasks}n

- Mỗi chương trình MapReduce là một công việc (job) được phân rã làm nhiều tác vụ độc lập (task) và các tác vụ này được phân tán trên các nodes khác nhau của cụm để thực thi
- Mỗi tác vụ được thực thi độc lập với các tác vụ khác để đạt được tính khả mở
 - Giảm truyền thông giữa các node máy chủ
 - Tránh phải thực hiện cơ chế đồng bộ giữa các tác vụ
- “Moving Computation is Cheaper than Moving Data”
 - Di chuyển tác vụ (KB, MB) tới Node thay vì
 - Di chuyển dữ liệu (TB, PB) tới Node tính toán

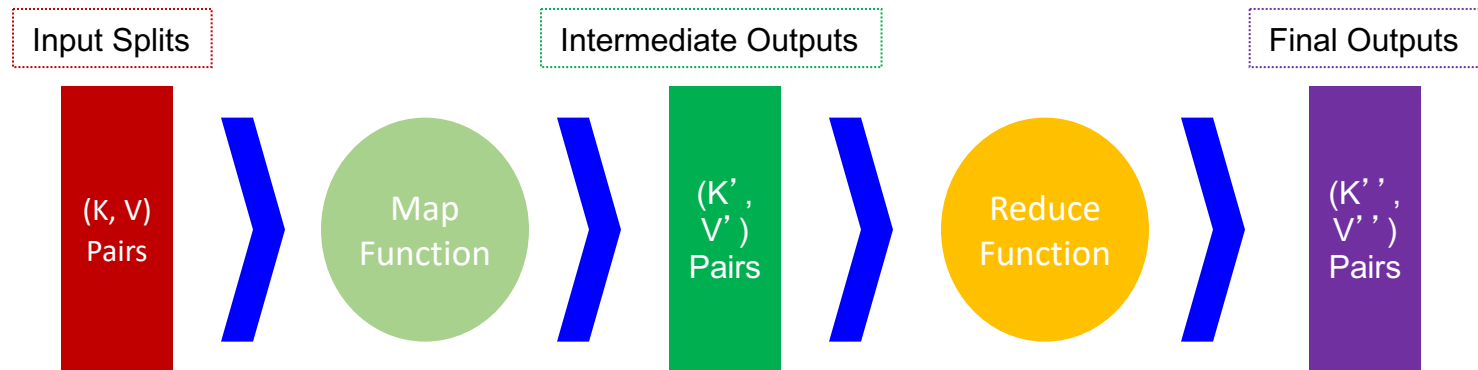
Dữ liệu cho MapReduce

- MapReduce trong môi trường Hadoop thường làm việc với dữ liệu đã có sẵn trên HDFS
- Khi thực thi, mã chương trình MapReduce được gửi tới các node đã có dữ liệu tương ứng



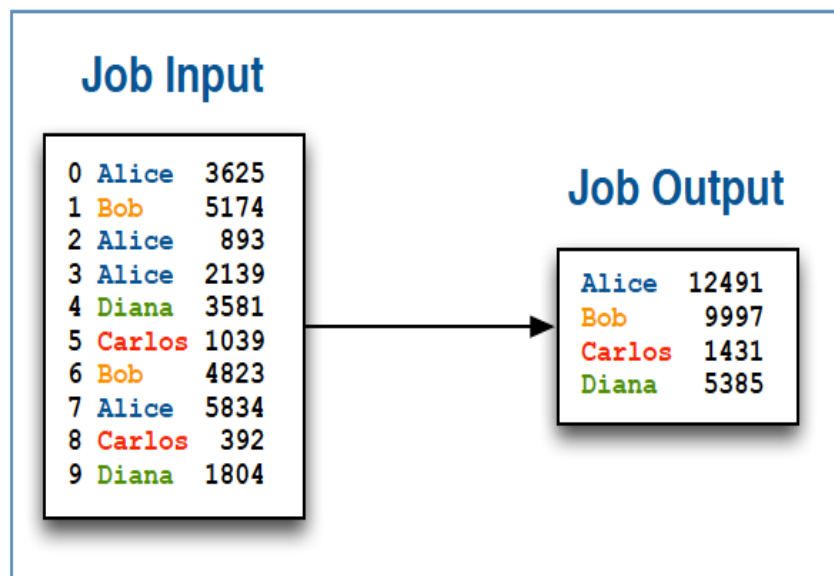
MapReduce

- Lập trình với MapReduce cần cài đặt 2 hàm Map và Reduce
- Trong chương trình MapReduce, dữ liệu được nhìn nhận như là các cặp khóa – giá trị (key – value)
- Các hàm Map và Reduce nhận đầu vào và trả về đầu ra các cặp (key – value)



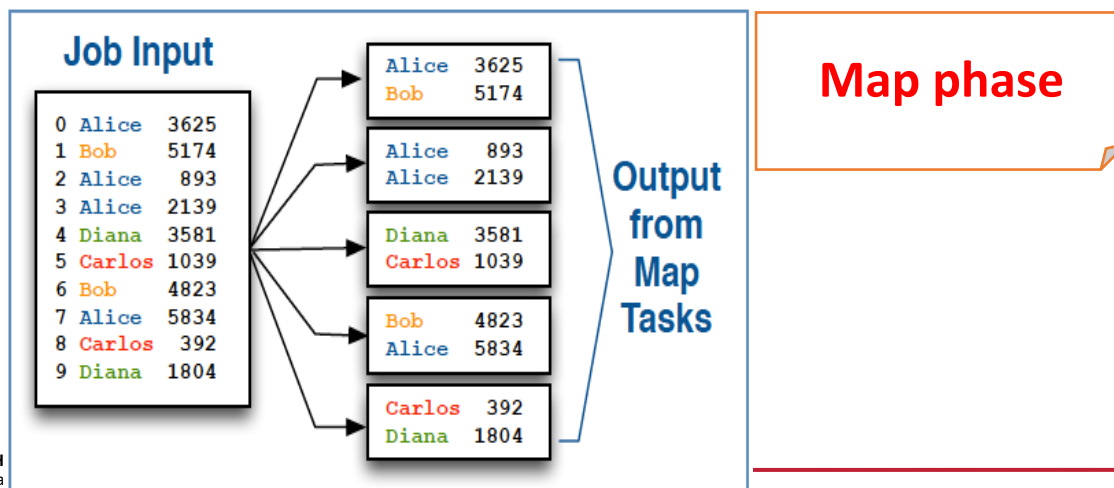
Ví dụ về MapReduce

- Đầu vào: tệp văn bản chứa thông tin về order ID, employee name, and sale amount
- Đầu ra : Doanh số bán (sales) theo từng nhân viên (employee)



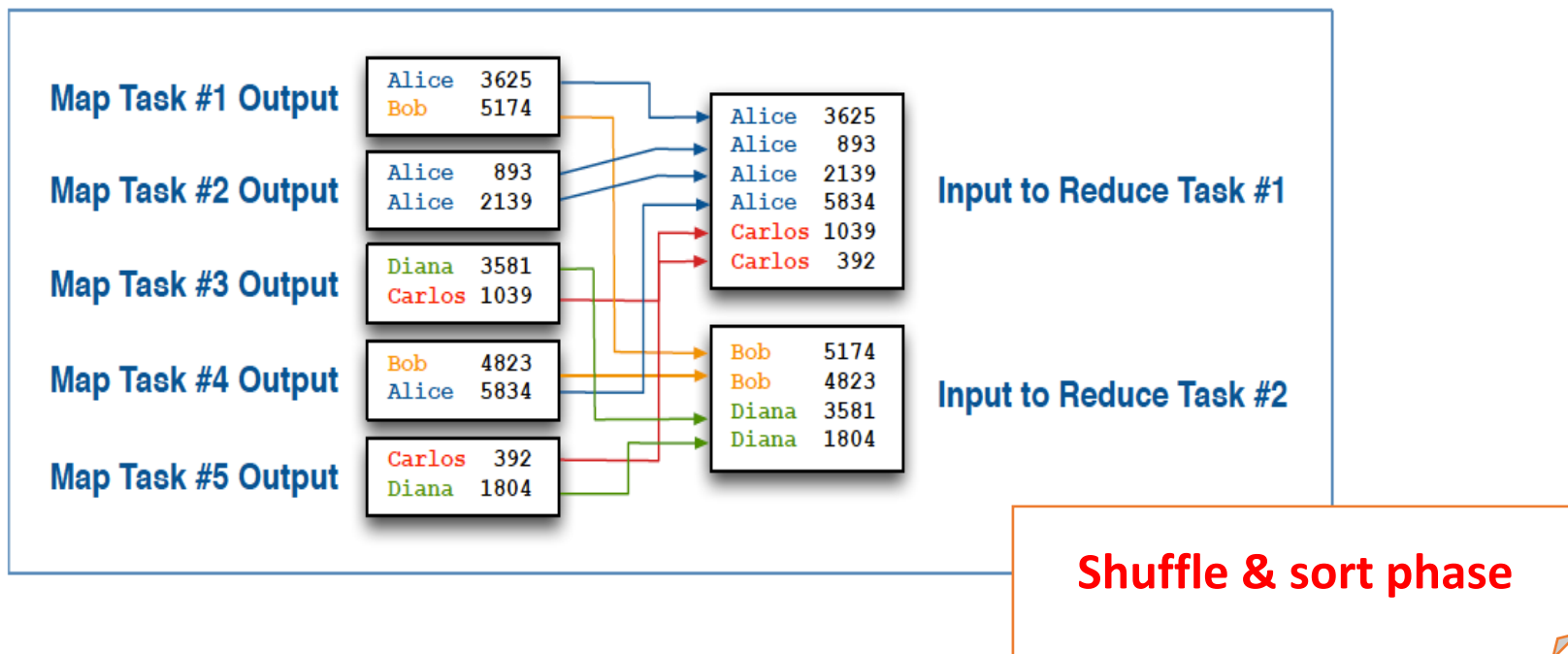
Bước Map

- Dữ liệu đầu vào được xử lý bởi nhiều tác vụ Mapping độc lập
 - Số lượng các tác vụ Mapping được xác định theo lượng dữ liệu đầu vào (~ số chunks)
 - Mỗi tác vụ Mapping xử lý một phần dữ liệu (chunk) của khối dữ liệu ban đầu
- Với mỗi tác vụ Mapping, Mapper xử lý lần lượt từng bản ghi đầu vào
 - Với mỗi bản ghi đầu vào (key-value), Mapper đưa ra 0 hoặc nhiều bản ghi đầu ra (key – value trung gian)
- Trong ví dụ này, tác vụ Mapping đơn giản đọc từng dòng văn bản và đưa ra tên nhân viên và doanh số tương ứng



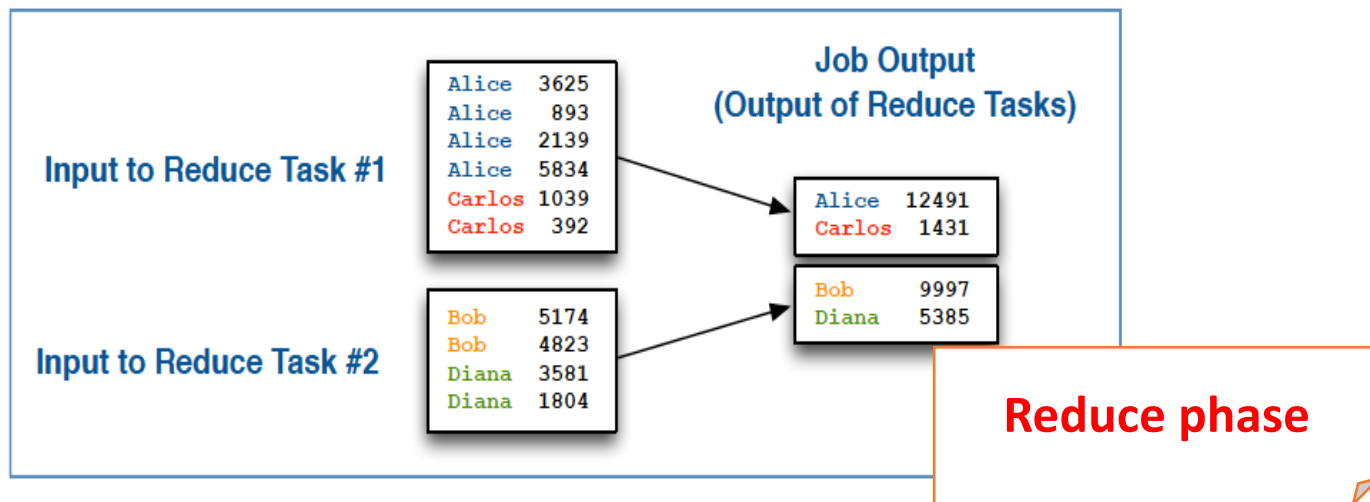
Bước shuffle & sort

- Hadoop tự động sắp xếp và gộp đầu ra của các Mappers theo các partitions
 - Mỗi partitions là đầu vào cho một Reducer

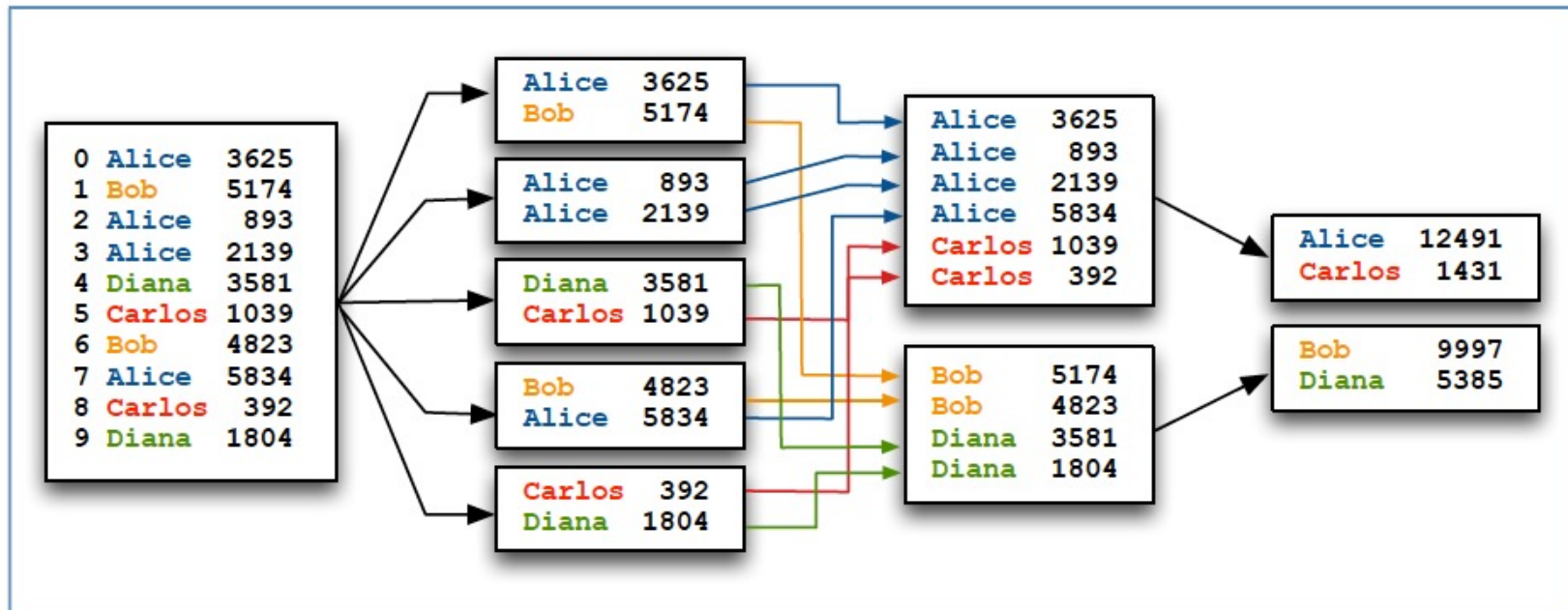


Bước Reduce

- Reducer nhận dữ liệu đầu vào từ bước shuffle & sort
 - Tất cả các bản ghi key – value tương ứng với một key được xử lý bởi một Reducer duy nhất
 - Giống bước Map, Reducer xử lý lần lượt từng key, mỗi lần với toàn bộ các values tương ứng
- Trong ví dụ, hàm reduce đơn giản là tính tổng doanh số cho từng nhân viên, đầu ra là các cặp key – value tương ứng với tên nhân viên – doanh số tổng

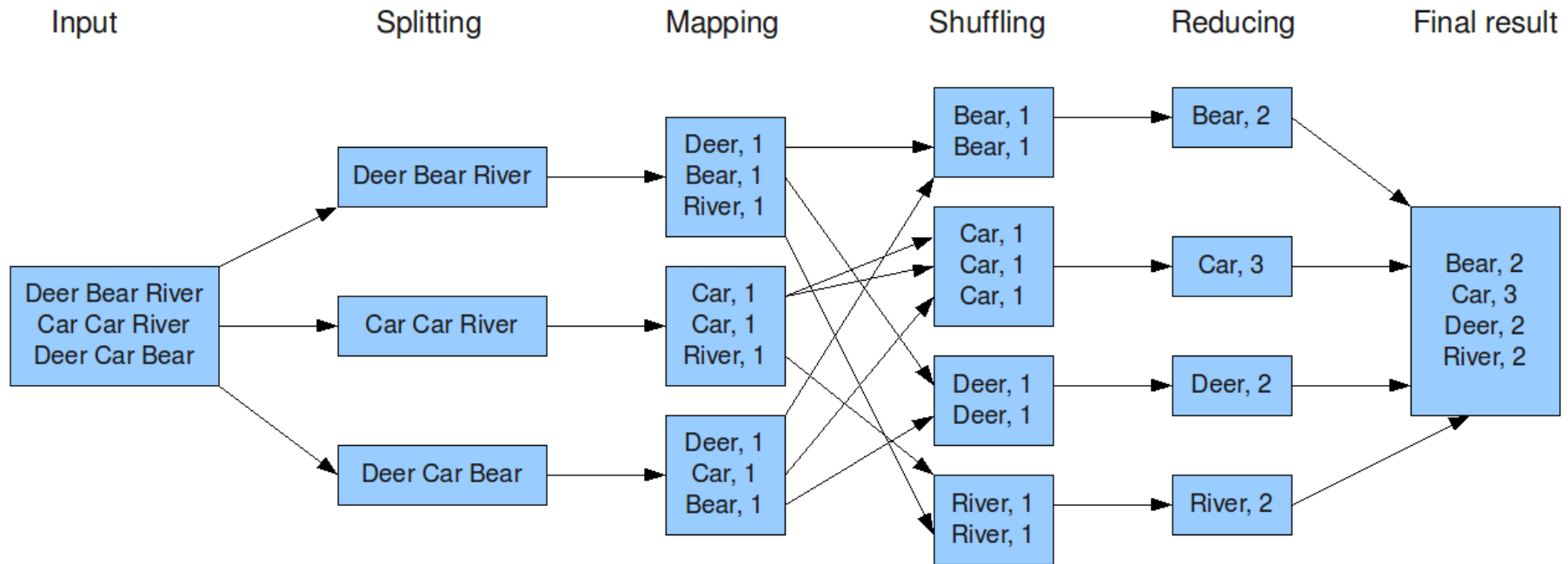


Luồng dữ liệu

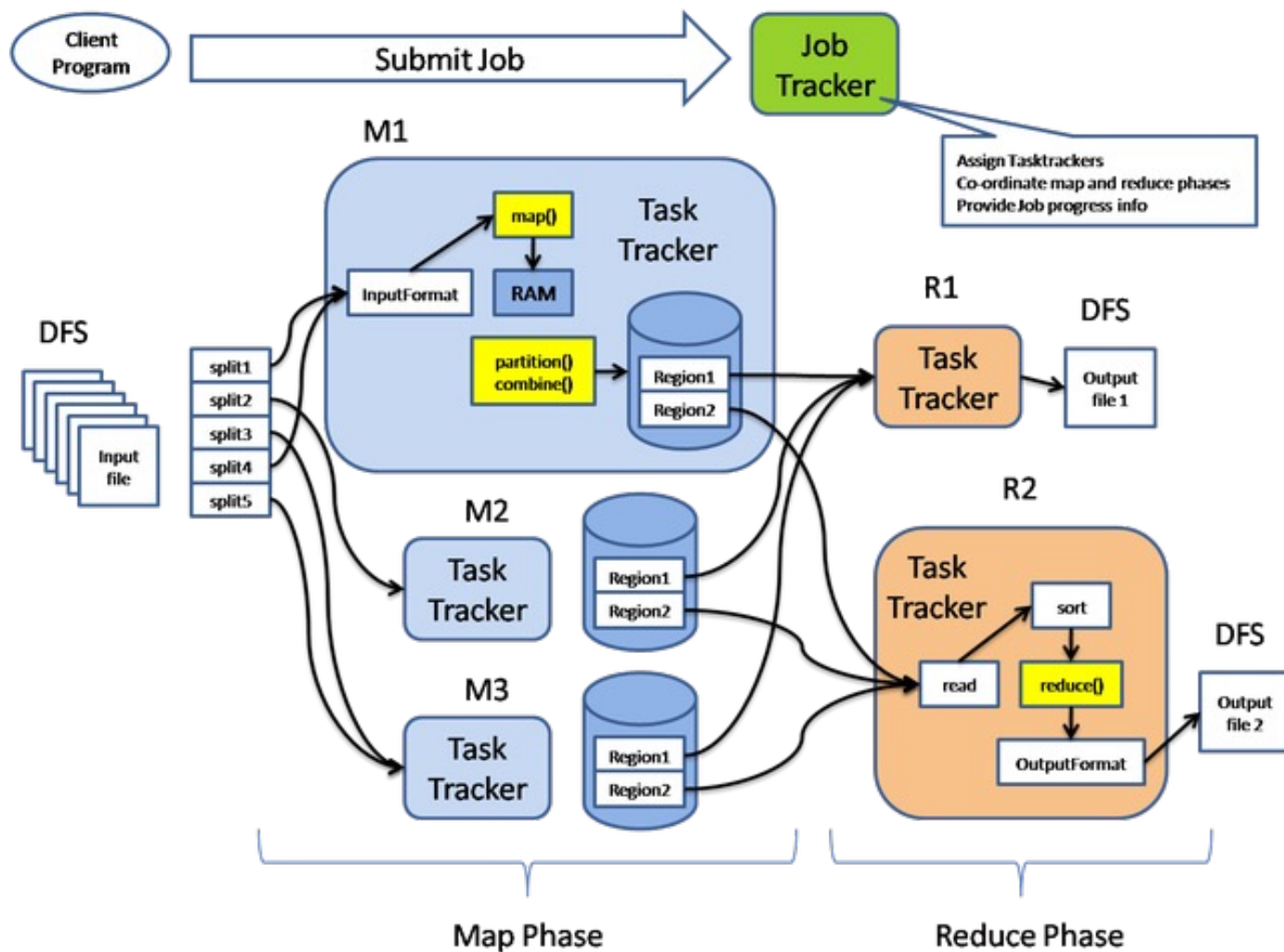


Bài toán Word Count

The overall MapReduce word count process



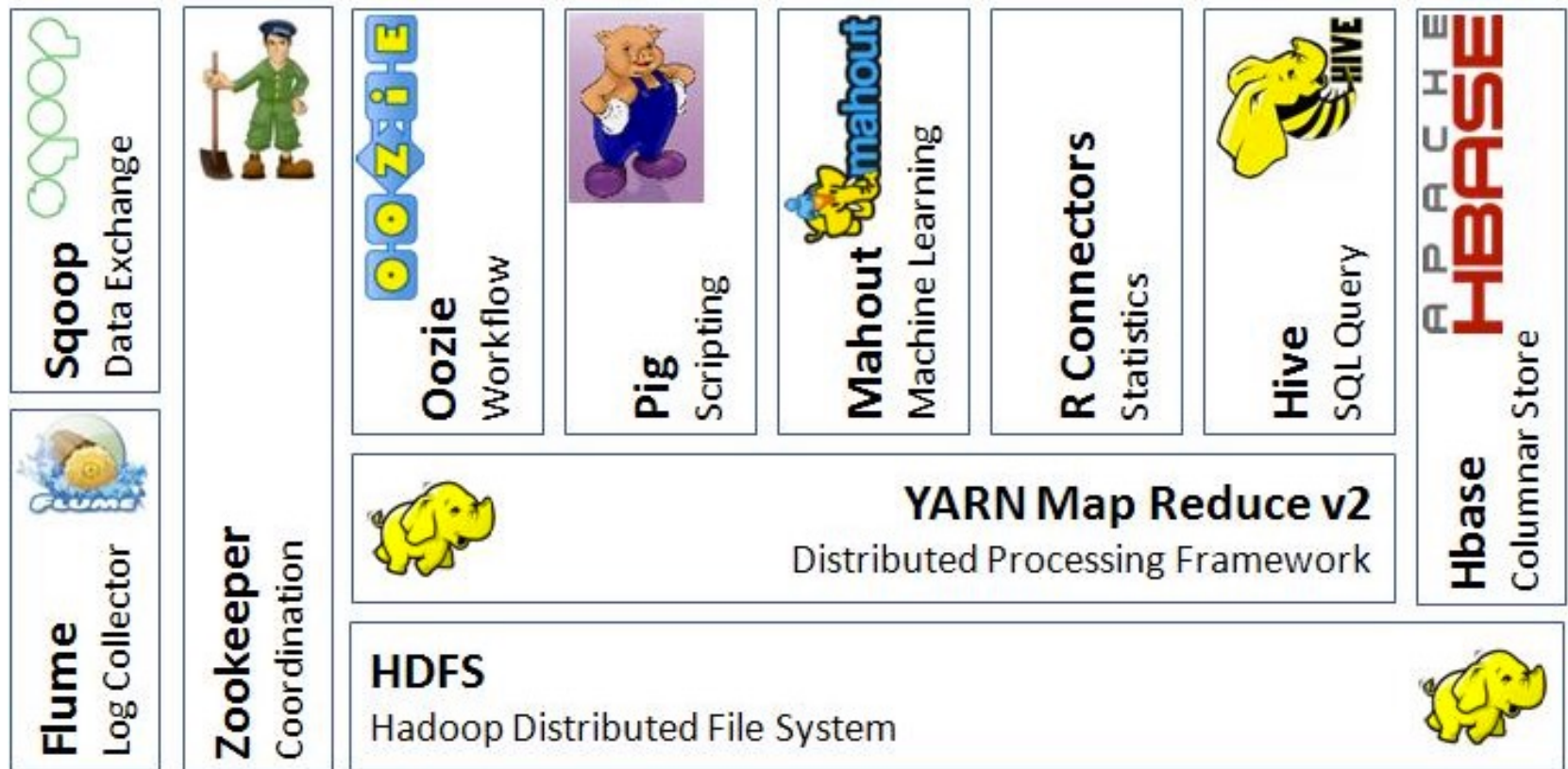
Vai trò của Job tracker và Task tracker



Các thành phần khác trong hệ sinh thái Hadoop

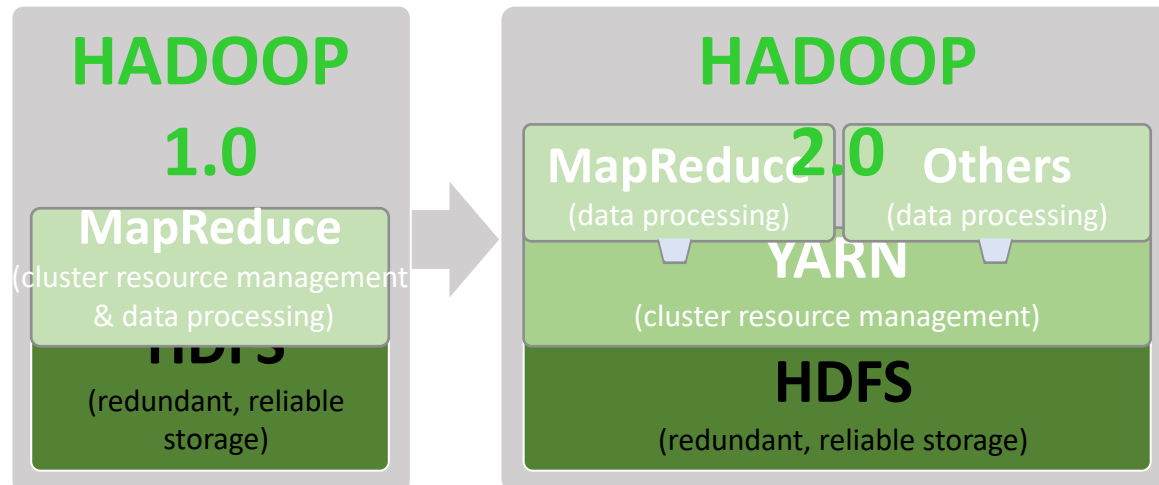
- Ngoài HDFS và MapReduce, hệ sinh thái Hadoop còn nhiều hệ thống, thành phần khác phục vụ
 - Phân tích dữ liệu
 - Tích hợp dữ liệu
 - Quản lý luồng

Bức tranh tổng thể hệ sinh thái Hadoop



YARN – Yet Another Resource Negotiator

- Nodes có tài nguyên là – bộ nhớ và CPU cores
- YARN đóng vai trò cấp phát lượng tài nguyên phù hợp cho các ứng dụng khi có yêu cầu
- YARN được đưa ra từ Hadoop 2.0
 - Cho phép MapReduce và non MapReduce cùng chạy trên 1 cụm Hadoop
 - Với MapReduce job, vai trò của job tracker được thực hiện bởi application tracker



Apache Pig

- Apache Pig cung cấp giao diện xử lý dữ liệu mức cao
 - Pig đặc biệt tốt cho các phép toán Join và Transformation
- Trình biên dịch của Pig chạy trên máy client
 - Biến đổi PigLatin script thành các jobs của MapReduce
 - Đệ trình các công việc này lên cụm tính toán



```
people = LOAD '/user/training/customers' AS (cust_id, name);
orders = LOAD '/user/training/orders' AS (ord_id, cust_id, cost);
groups = GROUP orders BY cust_id;
totals = FOREACH groups GENERATE group, SUM(orders.cost) AS t;
result = JOIN totals BY group, people BY cust_id;
DUMP result;
```

Apache Hive

- Cũng là một lớp trừu tượng mức cao của MapReduce
 - Giảm thời gian phát triển
 - Cung cấp ngôn ngữ HiveQL: SQL-like language
- Trình biên dịch Hive chạy trên máy client
 - Chuyển HiveQL script thành MapReduce jobs
 - Đệ trình các công việc này lên cụm tính toán



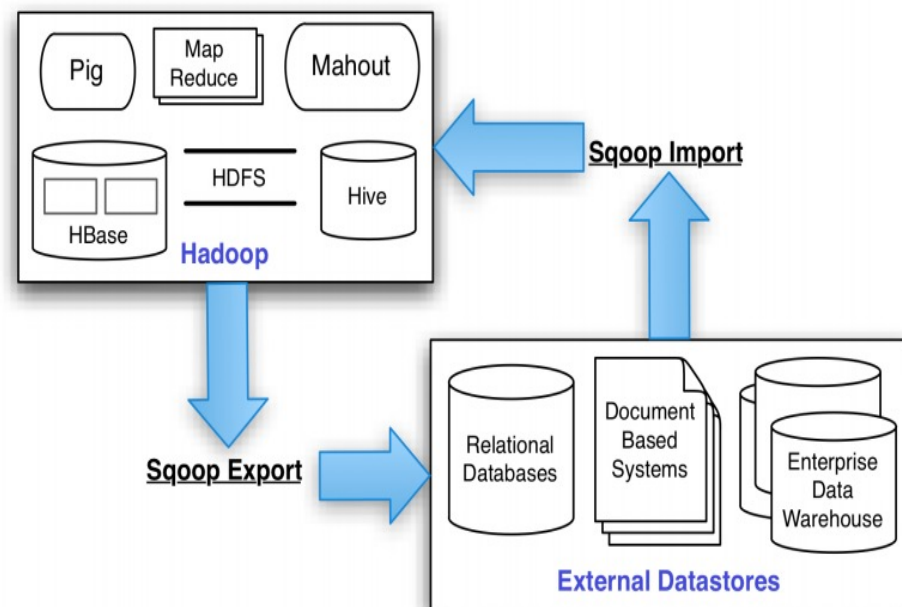
```
SELECT customers.cust_id, SUM(cost) AS total
FROM customers
JOIN orders
  ON customers.cust_id = orders.cust_id
GROUP BY customers.cust_id
ORDER BY total DESC;
```



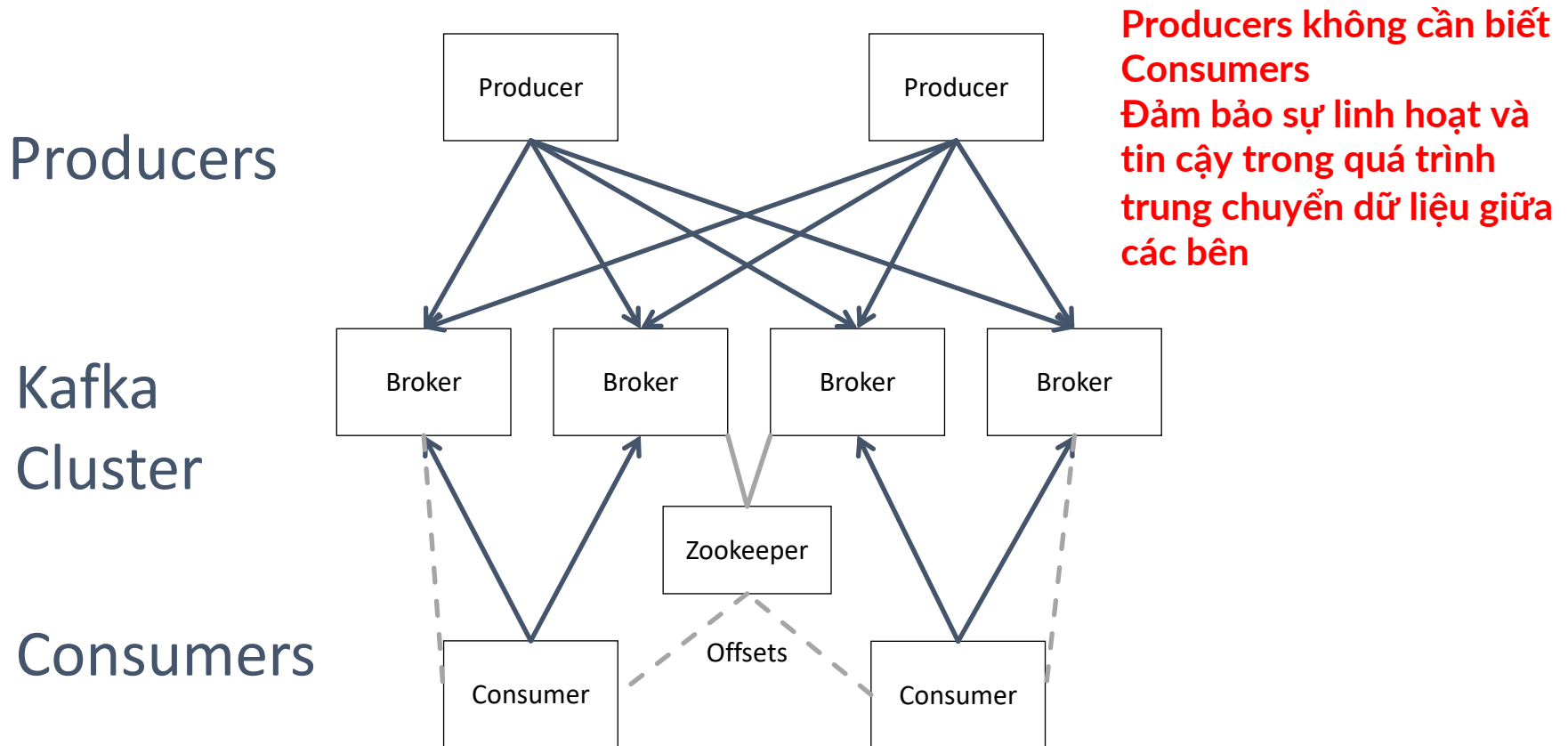
- HBase là một CSDL mở rộng phân tán, lưu trữ dữ liệu trên HDFS
 - Được xem như là hệ quản trị CSDL của Hadoop
- Dữ liệu được tổ chức về mặt logic là các bảng, bao gồm rất nhiều dòng và cột
 - Kích thước bảng có thể lên đến hàng Terabyte, Petabyte
 - Bảng có thể có hàng ngàn cột
- Có tính khả mở cao, đáp ứng bằng thông ghi dữ liệu tốc độ cao
 - Hỗ trợ hàng trăm ngàn thao tác INSERT mỗi giây (/s)
- Tuy nhiên về các chức năng thì còn rất hạn chế khi so sánh với hệ QTCSDL truyền thống
 - Là NoSQL : không có ngôn ngữ truy vấn mức cao như SQL
 - Phải sử dụng API để scan/ put/ get/ dữ liệu theo khóa

Apache Sqoop

- Sqoop là một công cụ cho phép trung chuyển dữ liệu theo khối từ Apache Hadoop và các CSDL có cấu trúc như CSDL quan hệ
- Hỗ trợ import tất cả các bảng, một bảng hay 1 phần của bảng vào HDFS
 - Thông qua Map only hoặc MapReduce job
 - Kết quả là 1 thư mục trong HDFS chứa các tập tin văn bản phân tách các trường theo ký tự phân tách (vd. , hoặc \t)
- Hỗ trợ export dữ liệu ngược trở lại từ Hadoop ra bên ngoài



Apache Kafka



Kafka cho phép phân tách mạch lạc các thành phần tham gia vào luồng dữ liệu

Apache Oozie

- Oozie là một hệ thống lập lịch luồng công việc để quản lý các công việc thực thi trên cụm Hadoop
- Luồng workflow của Oozie là đồ thị vòng có hướng (Directed Acyclical Graphs (DAGs)) của các khối công việc
- Oozie hỗ trợ đa dạng các loại công việc
 - Thực thi MapReduce jobs
 - Thực thi Pig hay Hive scripts
 - Thực thi các chương trình Java hoặc Shell
 - Tương tác với dữ liệu trên HDFS
 - Chạy chương trình từ xa qua SSH
 - Gửi nhận email



Apache Zookeeper

- Apache ZooKeeper là một dịch vụ cung cấp các chức năng phối hợp phân tán độ tin cậy cao
 - Quản lý thành viên trong nhóm máy chủ
 - Bầu cử leader
 - Quản lý thông tin cấu hình động
 - Giám sát trạng thái hệ thống
- Đây là các service lõi, tối quan trọng trong các hệ thống phân tán

